

深圳大学实验报告

课程名称 计算机游戏开发

项目名称 实验 2 游戏交互界面设计

学 院 数学科学学院

专 业 信息与计算科学（数学与计算机实验班）

指导教师 储 颖

报 告 人 王曦 学号 2021192010

实验时间 2024 年 04 月 22 日

提交时间 2024 年 04 月 22 日

教务处制

目录

一、实验目的与要求.....	3
二、实验内容与方法.....	3
三、实验步骤与过程.....	4
1. 编译游戏.....	4
1.1 生成项目.....	4
1.2 加入文件.....	4
1.3 修改代码.....	5
1.4 成功运行.....	5
2. 修改窗口标题.....	6
3. 摇杆功能扩展.....	7
3.1 问题引入.....	7
3.2 规范化.....	7
3.3 实现.....	7
4. 计分板.....	9
4.1 创建计分板.....	9
4.2 接住球加分.....	10
5. 游戏 Bug 修复.....	13
5.1 球越界.....	13
5.2 球一闪而过.....	14
5.3 摇杆判定范围.....	14
6. 游戏优化.....	16
6.1 游戏策划.....	16
6.2 设置 (Config.h, Config.cpp)	21
6.3 开始界面 (Login.h, Login.cpp)	21
6.4 时间、判定线、Note 类 (Note.h, Note.cpp)	23
6.5 关卡父类 (HelloWorldScene.h, HelloWorldScene.cpp)	26
6.6 关卡二类 (部分) (Level2.h, Level2.cpp)	28
6.7 update()函数 (Level2.h, Level2.cpp)	35
6.8 关卡三类 (部分) (Level3.h, Level3.cpp)	40
6.9 运行效果.....	41
四、实验结论或心得体会.....	42
7. 实验结论.....	42
7.1 编译游戏.....	42
7.2 修改标题.....	42
7.3 扩展摇杆功能.....	43
7.4 计分板.....	43
7.5 开始界面.....	43
7.6 重玩功能.....	44
7.7 增加关卡.....	45
7.8 优化.....	45
8. 实验心得.....	46
8.1 核心心得.....	46
8.2 其它心得.....	46

一、实验目的与要求

- 1.熟悉交互界面设计原理。
- 2.了解 Cocos2d-x 中的用户交互、触摸事件、碰撞检测机制。

二、实验内容与方法

1. 完成游戏编译 (15 分)

仿照实验一“英雄快跑”实验，将教材源码和素材文件复制到自己的项目中，成功编译并运行本次实验----“贪食豆”游戏。

2. 修改游戏显示名称 (5 分)

通过修改游戏代码，使自己的学号姓名(中文)替换原“MyGame”字样出现在标题栏左上角。

3. 增加摇杆上下移动功能 (5 分)

修改游戏代码，使贪食豆在屏幕范围内能上下左右移动。

4. 增加计分板功能 (5 分)

修改游戏代码，增加计分板功能。

5. 增加 UI 登录界面(含 Play 按钮) (10 分)

请自行下载素材，用 Cocos Studio UI Editor 设计个性化登录界面(含按钮)，并在项目中加载该登录界面，实现开始游戏(Play)功能。

6. 增加 Replay 按钮 (5 分)

在游戏结束时添加 Replay 按钮，实现重新开始游戏(Replay)功能。

7. 增加一个关卡 (5 分)

至少增加一个关卡，第二关要体现难度的递增以及游戏元素的丰富。

8. 游戏优化 (5 分)

自行发挥想象力，优化游戏功能。

9. 录制通关视频 (5 分)

录制游戏通关视频，上传至 BB 系统。

10. 完成实验报告 (40 分)

截图记录关键步骤，分析实验结果，撰写心得体会。

三、实验步骤与过程

(记录关键步骤/设计过程/设计结果的截图)

1. 编译游戏

1.1 生成项目

在 `cocos2d-x-3.17.2\tools\cocos2d-console\bin` 目录下，打开命令行，用命令

```
cocos new -l cpp EXP2
```

构建项目。如图 1.1 所示。

```
D:\cocos2d-x-3.17.2\tools\cocos2d-console\bin>cocos new -l cpp EXP2
> 拷贝模板到 D:\cocos2d-x-3.17.2\tools\cocos2d-console\bin\EXP2
> 拷贝 cocos2d-x ...
> 替换文件名中的工程名称, 'HelloCpp' 替换为 'EXP2'。
> 替换文件中的工程名称, 'HelloCpp' 替换为 'EXP2'。
> 替换工程的包名, 'org.cocos2dx.hellocpp' 替换为 'org.cocos2dx.EXP2'。
> 替换 Mac 工程的 Bundle ID, 'org.cocos2dx.hellocpp' 替换为 'org.cocos2dx.EXP2'。
> 替换 iOS 工程的 Bundle ID, 'org.cocos2dx.hellocpp' 替换为 'org.cocos2dx.EXP2'。
```

图 1.1: 命令行构建项目

1.2 加入文件

将 `Chapter05-Chapter12\源代码\CocosExample\Resources` 目录下的 `Chapter 10` 文件夹和 `fonts` 文件夹复制到 `EXP2\Resources` 目录下。如图 1.2 所示。

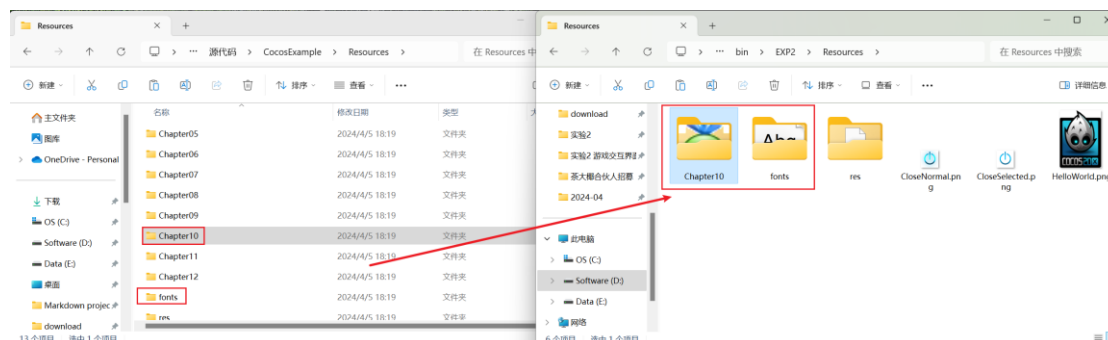


图 1.2: 替换资源

将 `Chapter05-Chapter12\源代码\CocosExample\Classes\Chapter10` 目录下的 `HelloWorldScene.cpp`、`HelloWorldScene.h`、`Joystick.cpp` 和 `Joystick.h` 复制到 `EXP1\Classes` 目录下。如图 1.3 所示。

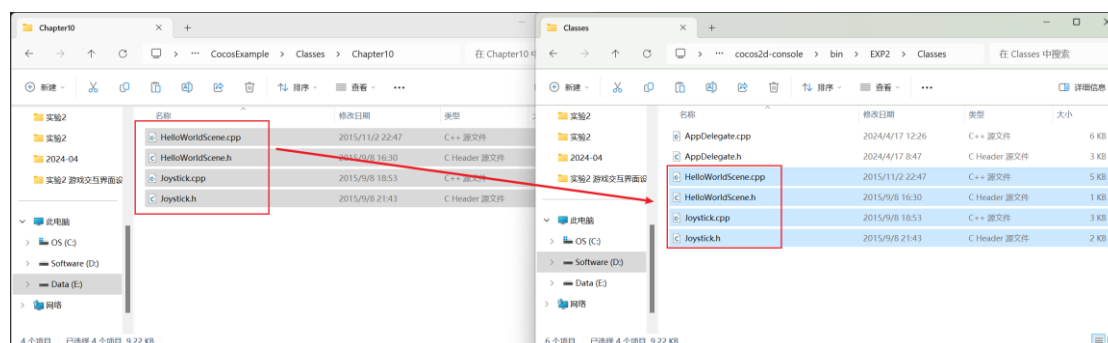


图 1.3: 替换源文件和头文件

2. 修改窗口标题

附件“cocos2d-x 中文输入问题解决方案.pdf”提供的 4 种实现中文输入的方法都可行。事实上，在 Windows 系统中有更简便的实现。

修改 AppDelegate.cpp 中 AppDelegate::applicationDidFinishLaunching() 函数，将学号和姓名加入标题中，用 u8 指定字符串采用 UTF-8 编码即可。代码如图 2.1 所示，运行结果如图 2.2 所示。

```
81 bool AppDelegate::applicationDidFinishLaunching() {
82     // initialize director
83     auto director = Director::getInstance();
84     auto glview = director->getOpenGLView();
85     if(!glview) {
86         #if (CC_TARGET_PLATFORM == CC_PLATFORM_WIN32) || (CC_TARGET_PLATFORM == CC_PL
87         glview = GLViewImpl::createWithRect(u8"2021192010王曦 EXP2", cocos2d:
88     #else
89         glview = GLViewImpl::create("EXP2");
90     #endif
91     director->setOpenGLView(glview);
92 }
```

图 2.1: 修改标题的代码

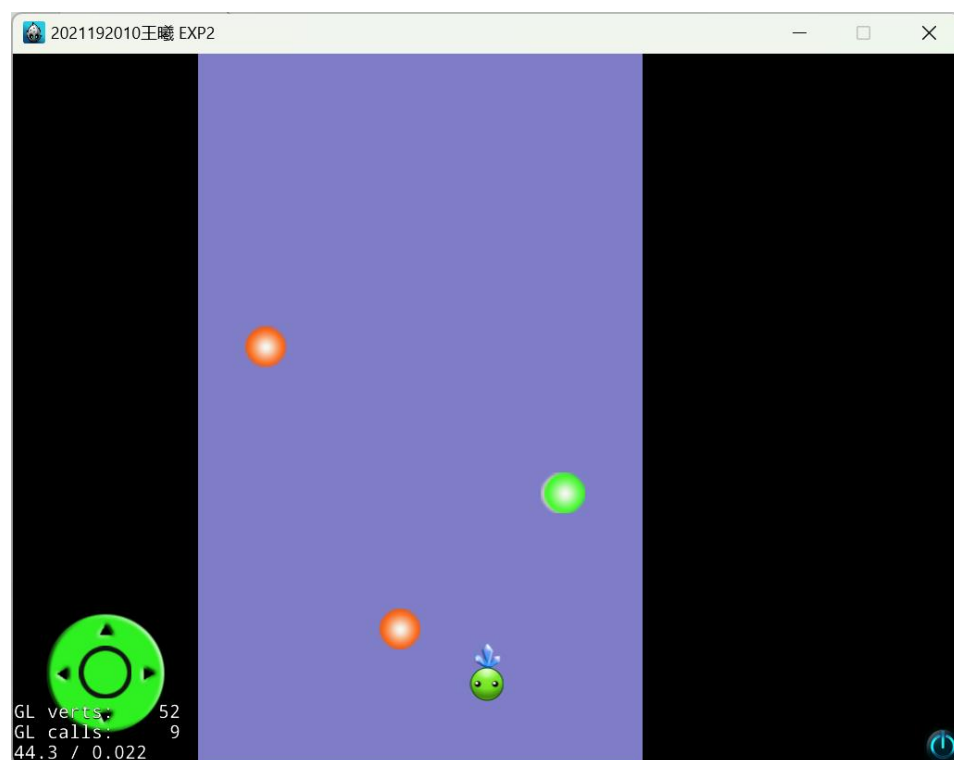


图 2.2: 修改标题的结果

3. 摇杆功能扩展

3.1 问题引入

框架代码已实现贪食豆的左右移动，下面实现上下移动。

观察 Joystick.cpp 中获取摇杆方向的实现，发现它使用摇杆的当前位置在中心点的左方或右方判断方向。如图 3.1 所示。

```
56      // 获取摇杆当前方向
57      Joystick_dir Joystick::getDirection()
58      {
59          if ((m_currentPoint - m_centerPoint).x > 0)
60          {
61              return Joystick_dir::_RIGHT;
62          } if ((m_currentPoint - m_centerPoint).x < 0)
63          {
64              return Joystick_dir::_LEFT;
65          }
66          return Joystick_dir::_STOP;
67      }
```

图 3.1: Joystick.cpp 中获取摇杆方向的实现

若在此基础上引入上下移动，则往斜向拉动摇杆时会出现方向的优先级问题，如向左上方拉动时应判定为向左还是向上。为避免该问题，下面将摇杆的功能修改为根据拉动方向 360° 地移动贪食豆。

3.2 规范化

因原代码中游戏面板尺寸的参数直接写在函数的参数中，不易修改和扩展，故先将它们独立为全局变量。在 HelloWorldScene.h 中定义上述全局变量。如图 3.2 所示。

```
7      const int BOARD_WIDTH = 480;
8      const int BOARD_WIDTH_BIAS = 200;
9      const int BOARD_HEIGHT = 768;
```

图 3.2: 在 HelloWorldScene.h 中定义上述全局变量

修改 HelloWorldScene.cpp 中 HelloWorld::init() 函数中添加颜色层的尺寸参数。如图 3.3 所示。

```
30      //添加颜色层
31      auto colorLayer = LayerColor::create(Color4B(128, 125, 200, 255), BOARD_WIDTH, BOARD_HEIGHT);
32      colorLayer->setPosition(Vec2(BOARD_WIDTH_BIAS, 0));
```

图 3.3: 修改 HelloWorldScene.cpp 中 HelloWorld::init() 函数中的尺寸参数

3.3 实现

为实现摇杆的功能修改为根据拉动方向 360° 地移动贪食豆，先摒弃 Joystick.h 中摇杆移动方向的枚举类型，并将 Joystick::getDirection() 函数的返回值类型修改为 Vec2。如图 3.2 所示。

```

9  |  /*enum Joystick_dir
10 |  {
11 |      _LEFT,
12 |      _RIGHT,
13 |      _STOP
14 |  };*/
15 |
16 |  class Joystick : public Layer
17 |  {
18 |  public:
19 |      Joystick();
20 |      ~Joystick();
21 |
22 |      // 创建摇杆, aPoint是摇杆中心 aRadius是摇杆半径 aJsSpr
23 |      static Joystick* create(Vec2 aPoint, float aRadius, cha
24 |
25 |      // 获取摇杆方向
26 |      // Joystick_dir getDirection();
27 |      Vec2 getDirection();

```

图 3.4: 修改 Joystick::getDirection()函数的返回值类型

修改 Joystick.cpp 中 Joystick::getDirection()函数的实现, 返回摇杆移动方向的方向向量, 注意单位化。如图 3.3 所示。

```

57 |  // 获取摇杆移动方向的方向向量
58 |  // Joystick_dir Joystick::getDirection()
59 |  Vec2 Joystick::getDirection()
60 |  {
61 |      /*if ((m_currentPoint - m_centerPoint).x > 0)
62 |      {
63 |          return Joystick_dir::_RIGHT;
64 |      } if ((m_currentPoint - m_centerPoint).x < 0)
65 |      {
66 |          return Joystick_dir::_LEFT;
67 |      }
68 |      return Joystick_dir::_STOP;*/
69 |
70 |      return (m_currentPoint - m_centerPoint).getNormalized();
71 |  }

```

图 3.5: 修改 Joystick::getDirection()函数的实现

修改 HelloWorldScene.cpp 的 HelloWorld::update()函数中控制贪食豆移动的实现, 此处移动的距离与拉杆力度成正比。计算预期目标点后, 再将其限制在区域内。如图 3.6 所示。

```

129 |  auto direction = m_joystick->getDirection(); // 摇杆移动方向
130 |  auto destination = bean->getPosition() + direction * m_joystick->getVelocity() * 0.15; // 预期目标点
131 |  auto beanSize = bean->getContentSize(); // 贪食豆大小
132 |  destination.x = std::max(destination.x, BOARD_WIDTH_BIAS + beanSize.width / 2);
133 |  destination.x = std::min(destination.x, BOARD_WIDTH_BIAS + BOARD_WIDTH - beanSize.width / 2);
134 |  destination.y = std::max(destination.y, 0 + beanSize.height / 2);
135 |  destination.y = std::min(destination.y, BOARD_HEIGHT - beanSize.height / 2);
136 |  bean->setPosition(destination);

```

图 3.6: HelloWorldScene.cpp 的 HelloWorld::update()函数中控制贪食豆移动的实现
运行效果见【附件】贪食豆.mp4。

4. 计分板

4.1 创建计分板

同【实验 1】的“Game Over”和“Success”的提示信息的创建方法，用一个标签作为计分板。

先在 HelloWorldScene.h 中定义标签的 BM 字体的路径和计分板的 Tag，其中指定计分板的 Tag 方便后续操作计分板。如图 4.1 所示。

```
22 // 标签字体
23 const std::string TIPS_FNT_PATH = "fonts/futura-48.fnt";
24
25 // 计分板
26 const int SCOREBOARD_TAG = 1;
```

图 4.1: 在 HelloWorldScene.h 中定义标签的 BM 字体的路径和计分板的 Tag

在 HelloWorldScene.h 中声明、在 HelloWorldScene.cpp 中定义变量 score 和函数 HelloWorld::modifyScore(_score)，用于将当前的分数修改为_score。如图 4.2 和图 4.3 所示。

```
48 // 更新计分板函数
49 void modifyScore(int _score);
50 private:
51     Sprite* bean;
52     Size visibleSize;
53     Joystick* m_joystick;
54     Vector<Sprite*> ballVector;
55
56     int score;
```

图 4.2: HelloWorldScene.h 中变量和函数的声明

```
20 void HelloWorld::modifyScore(int _score) {
21     // 清除旧计分板
22     auto trash = this->getChildByTag(SCOREBOARD_TAG);
23     if (trash) {
24         this->removeChildByTag(SCOREBOARD_TAG);
25     }
26
27     score = _score;
28     auto scoreboard = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(score));
29     scoreboard->setPosition(Vec2(100, visibleSize.height / 2));
30     scoreboard->setTag(SCOREBOARD_TAG);
31     this->addChild(scoreboard, 1);
32 }
```

图 4.3: HelloWorldScene.cpp 中函数 HelloWorld::modifyScore(_score)的定义
运行结果如图 4.4 所示。



图 4.4: 增加计分板

4.2 接住球加分

注意到 HelloWorld.cpp 的 HelloWorld::init() 函数通过 schedule_selector() 定时器指定三种球的出现频率: 白球 > 橙球 > 绿球, 如图 4.5 所示。故可设置得分白球 < 橙球 < 绿球, 不妨设得分分别为 1、2、3。

```

72 // 随机掉落 3 种球
73 this->schedule(schedule_selector(HelloWorld::addBall1), 1.0f);
74 this->schedule(schedule_selector(HelloWorld::addBall2), 2.0f);
75 this->schedule(schedule_selector(HelloWorld::addBall3), 3.0f);
76 this->scheduleUpdate();

```

图 4.5: schedule_selector() 定时器指定三种球的出现频率

为方便实现, 在 HelloWorldScene.h 中取球的 Tag 与得分的偏移量为 10, 并取白球、橙球、绿球的 Tag 分别为 11、12、13, 则接到球的得分即 (球的 Tag - BALL_TAG_BIAS)。如图 4.6 所示, 其中还指定了 3 种球的贴图的路径。

```

13 // 3 种球
14 const std::string BALL1_PATH = "Chapter10/ball1.png";
15 const std::string BALL2_PATH = "Chapter10/ball2.png";
16 const std::string BALL3_PATH = "Chapter10/ball3.png";
17 const int BALL_TAG_BIAS = 10; // 球的 Tag 与得分的偏移量
18 const int BALL1_TAG = 11;
19 const int BALL2_TAG = 12;
20 const int BALL3_TAG = 13;

```

图 4.6: 3 种球的路径和 Tag

此处修改了 3 种球的贴图的文件名, 以与上述变量名对应。如图 4.7 所示。若修改后运

行发现所有白球都变成橙球，则删除 EXP2\proj.win32\Debug.win32 文件夹后重新生成解决方案即可。



图 4.7: 修改 3 种球的贴图的文件名

在 HelloWorldScene.cpp 的 HelloWorld::addBall1()、HelloWorld::addBall2()、HelloWorld::addBall3()函数中指定贴图的路径，并为球加 Tag。此处以 HelloWorld::addBall1()函数为例。如图 4.8 所示。

```
83 void HelloWorld::addBall1(float dt) {
84     auto ball1 = Sprite::create(BALL1_PATH); // 使用图片创建小球
85     ball1->setPosition(Point(207 + rand() % 460, BOARD_HEIGHT)); // 设置位置
86     ball1->setTag(BALL1_TAG);
87     this->addChild(ball1, 5);
88
89     this->ballVector.pushBack(ball1); // 将小球放进 Vector 数组
90
91     auto moveTo = MoveTo::create(rand() % 5, Point(ball1->getPositionX(),
92     auto actionDone = CallFunc::create(CC_CALLBACK_0(HelloWorld::removeBall, this, ball1));
93     auto sequence = Sequence::create(moveTo, actionDone, nullptr);
94     ball1->runAction(sequence); // 执行动作
95 }
```

图 4.8: HelloWorld::addBall1()函数

在 HelloWorldScene.cpp 的 HelloWorld::update()函数的碰撞检测中，先得分，再清除小球。如图 4.9 所示。

```
130 void HelloWorld::update(float dt) {
131     for (auto ball : ballVector)
132     {
133         // 进行碰撞检测
134         if (bean->getBoundingBox().intersectsRect(ball->getBoundingBox()))
135         {
136             // 得分
137             int tag = ball->getTag();
138             modifyScore(score + tag - BALL_TAG_BIAS);
139
140             // 删除球
141             auto actionDown = CallFunc::create(CC_CALLBACK_0(HelloWorld::removeBall, this, ball));
142             ball->runAction(actionDown);
143         }
144     }
```

图 4.9: 在 HelloWorldScene.cpp 的 HelloWorld::update()函数中得分
得分效果如图 4.10 所示。

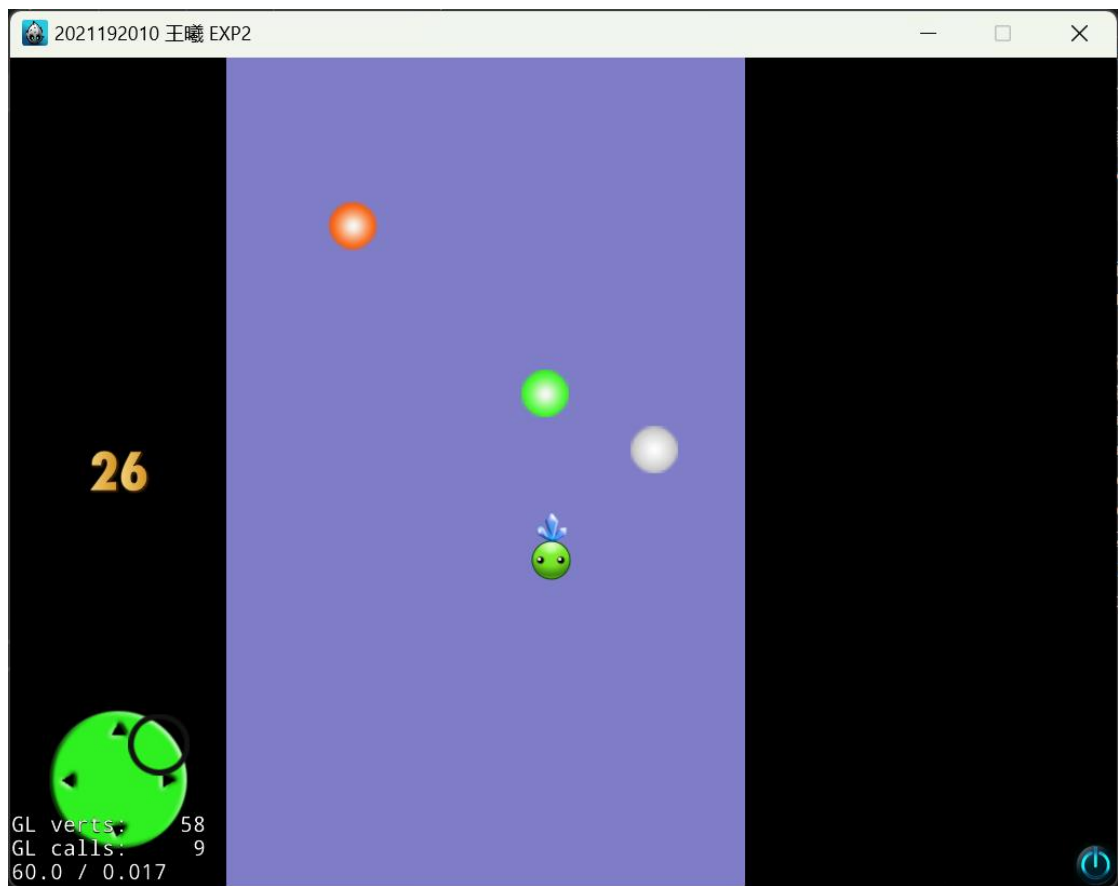


图 4.10: 得分效果

运行效果见【附件】贪食豆.mp4。

5. 游戏 Bug 修复

5.1 球越界

运行过程中会出现球擦边而过的情况。如图 5.1 所示。



图 5.1: 球擦边而过

检查发现球的贴图大小为 44 x 44，而球随机生成的锚点位置在 [207, 666] 范围内，故会发生球超出边界的情况。如图 5.2 所示。

```
83 void HelloWorld::addBall1(float dt) {  
84     auto ball1 = Sprite::create(BALL1_PATH); // 使用图片创建小球  
85     ball1->setPosition(Point(207 + rand() % 460, BOARD_HEIGHT)); // 设置小球  
86     ball1->setTag(BALL1_TAG);  
87     this->addChild(ball1, 5);  
88  
89     this->ballVector.pushBack(ball1); // 将小球放进 Vector 数组  
90  
91     auto moveTo = MoveTo::create(rand() % 5, Point(ball1->getPositionX(), -  
92     auto actionDone = CallFunc::create(CC_CALLBACK_0(HelloWorld::removeBall  
93     auto sequence = Sequence::create(moveTo, actionDone, nullptr);  
94     ball1->runAction(sequence); // 执行动作  
95 }
```

图 5.2: 球随机生成的锚点位置范围

为修复该 Bug, 将球随机生成的锚点位置限制在 $[\text{BOARD_WIDTH_BIAS} + \text{ball.width} / 2, \text{BOARD_WIDTH_BIAS} + \text{BOARD_WIDTH} - \text{ball.width} / 2]$ 范围内即可。设上述区间长度为 $\text{len} = \text{BOARD_WIDTH} - \text{ball.width} + 1$, 则锚点的 x 坐标取 $\text{rand}() \% \text{len} +$

BOARD_WIDTH_BIAS + ball.width / 2 即可。同理修改 HelloWorld::addBall2() 函数、HelloWorld::addBall3()函数。如图 5.3 所示。

```
83 void HelloWorld::addBall1(float dt){
84     auto ball1 = Sprite::create(BALL1_PATH); // 使用图片创建小球
85     int width = ball1->getContentSize().width;
86     ball1->setPosition(Point(
87         rand() % (BOARD_WIDTH - width + 1) + BOARD_WIDTH_BIAS + width / 2,
88         BOARD_HEIGHT)); // 设置小球的初始位置, 这里在x方向使用了随机函数rand使得小球
89     ball1->setTag(BALL1_TAG);
90     this->addChild(ball1, 5);
91
92     this->ballVector.pushBack(ball1); // 将小球放进 Vector 数组
93
94     auto moveTo = MoveTo::create(rand() % 5, Point(ball1->getPositionX(), -10));
95     auto actionDone = CallFunc::create(CC_CALLBACK_0(HelloWorld::removeBall, this));
96     auto sequence = Sequence::create(moveTo, actionDone, nullptr);
97     ball1->runAction(sequence); // 执行动作
98 }
```

图 5.3: 修改随机生成的球的锚点的 x 坐标

修改后球至多与边界相切, 不出现越界的情况。

运行效果见【附件】贪食豆.mp4。

5.2 球一闪而过

运行过程中有概率看到上边界有球一闪而过。此处无法通过截图展示, 不是因为笔者手速不够快, 而是就算截到了也无法体现它是一闪而过的。

检查发现 HelloWorldScene.cpp 的 HelloWorld::addBall1()函数中指定动作完成时间的参数可能取 0, 即球花费 0 s 下落, 亦即球出现后一闪而过。如图 5.4 所示。

```
83 void HelloWorld::addBall1(float dt){
84     auto ball1 = Sprite::create(BALL1_PATH); // 使用图片创建小球
85     int width = ball1->getContentSize().width;
86     ball1->setPosition(Point(
87         rand() % (BOARD_WIDTH - width + 1) + BOARD_WIDTH_BIAS + width / 2,
88         BOARD_HEIGHT)); // 设置小球的初始位置, 这里在x方向使用了随机函数rand使得小球
89     ball1->setTag(BALL1_TAG);
90     this->addChild(ball1, 5);
91
92     this->ballVector.pushBack(ball1); // 将小球放进 Vector 数组
93
94     auto moveTo = MoveTo::create(rand() % 5, Point(ball1->getPositionX(), -10));
95     auto actionDone = CallFunc::create(CC_CALLBACK_0(HelloWorld::removeBall, this));
96     auto sequence = Sequence::create(moveTo, actionDone, nullptr);
97     ball1->runAction(sequence); // 执行动作
98 }
```

图 5.4: HelloWorld::addBall1()函数中指定动作完成时间的参数可能取 0

为修复该 Bug, 只需给时间加上一个正常数即可, 此处以 + 1 为例。同理修改 HelloWorld::addBall2()函数、HelloWorld::addBall3()函数。如图 5.5 所示。

```
94     auto moveTo = MoveTo::create(rand() % 5 + 1, Point(ball1->getPositionX(), -10)); // 移动动作
```

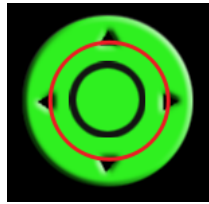
图 5.5: 给时间加上一个正常数

修改后球不再一闪而过。

运行效果见【附件】贪食豆.mp4。

5.3 摇杆判定范围

运行过程中发现，鼠标点击图 5.6 所示摇杆的红圈区域内有效果，但点击红圈区域外无效果，不符合常理。



检查 Joystick.cpp 的 Joystick::onTouchBegan() 函数中鼠标点击的判定，猜测 m_radius 取值过小。如图 5.7 所示。

```
85  bool Joystick::onTouchBegan(Touch *pTouch, Event *pEvent)
86  {
87      auto touchPoint = pTouch->getLocation();
88      if (touchPoint.getDistance(m_centerPoint) > m_radius) {
89          return false;
90      }
91      m_currentPoint = touchPoint;
92      return true;
93  }
```

图 5.7: Joystick.cpp 的 Joystick::onTouchBegan() 函数

检查 HelloWorldScene.cpp 中 HelloWorld::init() 函数中摇杆的创建，发现第二个参数（摇杆半径）取值过小。如图 5.8 所示。

```
68  // 摇杆
69  m_joystick = Joystick::create(Vec2(100, 100), 50.0f, "Chapter10/j-btn.png", "Chapter10/j-bg.png");
70  this->addChild(m_joystick, 4);
```

图 5.8: HelloWorldScene.cpp 中 HelloWorld::init() 函数中摇杆的创建

二分调参，合理的摇杆半径取值约 66.0f。

修改后点击摇杆区域内的所有区域都有效。

运行效果见【附件】贪食豆.mp4。

6. 游戏优化

6.1 游戏策划

6.1.1 概述

不同颜色的豆子下落类似于不同的 **note** 掉落，容易想到将《贪食豆》改造成音游。后续将其融入周深的元素，而周深的外号为“卡布”，而玩家在音游中的目标是流畅、准确地打到所有 **note**，故将新游戏称为《卡布不卡》。值得注意的是，这个名称正着读和反着读一样。

6.1.2 玩法

《贪食豆》中的贪食豆、下落的豆子对应《卡布不卡》中的周深的头、下落的音符。

音游中玩家需在 **note** 恰落到判定线上时进行打击，且传统音游中玩家只能在判定线附近打击 **note**，即只有左右移动，没有上下移动，而《贪食豆》中贪食豆可在任意位置接下落的豆子，且贪食豆可朝任意方向移动。

为适配音游的玩法，规定《卡布不卡》中：

- (1) 音符只能在判定线附近被周深接住。
- (2) 类似于 **Phigros**，判定线会上下移动，玩家需通过摇杆操控周深上下移动以在合适的位置接住音符。

根据玩家接 **note** 的时机是否准确，将每个 **note** 的命中状态分为：**Perfect**（准确）、**Good**（好，即没那么准确地接到了）、**Bad**（坏，即在错误的时间接到了）、**Miss**（错过，即没有接到）。值得注意的是，**note** 被判定为 **Bad** 后仍会继续下落至判定线，若玩家在判定线附近接到，也算一次 **Perfect** 或 **Good**；若玩家在判定线处未接到，则算一次 **Miss**。

《卡布不卡》保留了《贪食豆》中 **note** 随机掉落、玩家通过摇杆操控周深的头的移动，且《卡布不卡》中不设 **note** 落下的轨道（即 **key**），为平衡难度，《卡布不卡》中：

- (1) 稍加宽相邻 **note** 间出现的时间间隔。
- (2) 取合适的摇杆力度使得绝大部分时候相邻两 **note** 下落的时间间隔足以让周深的头从面板的最左侧移动到最右侧。
- (3) 加宽 **note** 的贴图，即玩家接到 **note** 的边缘也算接到。本次实验将 **note** 设置为长条形。如图 6.1.1 所示。

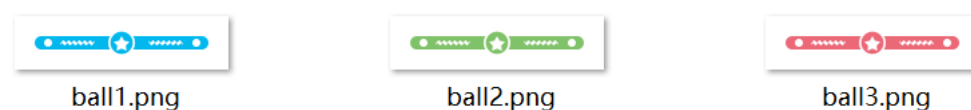


图 6.1.1: 3 种长条形 **note**

6.1.3 评分机制

设 **note** 的高度为 **height**，玩家接到 **note** 时 **note** 到判定线的有向距离为 **dis**，则 **note** 的命中状态的判断标准和得分为：

- (1) **Perfect**: $|\text{dis}| \leq \text{height} / 2$ ，得分 + 2。
- (2) **Good**: $|\text{dis}| \leq \text{height}$ ，得分 + 1。
- (3) **Bad**: $\text{dis} > 0$ 且 $\text{dis} > \text{height}$ ，得分 -1。
- (4) **Miss**: $\text{dis} < 0$ 且 $\text{dis} < -\text{height}$ ，不得分。

全曲结束后，根据玩家取得 **Perfect**、**Good** 的 **note** 数占歌曲总 **note** 数的比例打分。若玩家所有 **note** 都为 **Perfect**，则评价为 **All Perfect**（简称 **AP**）。

6.1.4 关卡设置

《卡布不卡》设 3 关，难度递增。所有关卡中，**note** 在随机时间、随机位置生成，并以

随机速度下落。玩家在规定时间内达到一定的分数。关卡设置如下：

- (1) 第 1 关：背景音乐《人是_》，有静止的判定线，玩家接到 note 即判 Perfect。
- (2) 第 2 关：背景音乐《Intro》，有静止的判定线，玩家接到 note 按 6.1.3 评分机制中的规则判命中状态。
- (3) 第 3 关：背景音乐《蜃楼》，有运动的判定线，玩家接到 note 按 6.1.3 评分机制中的规则判命中状态。

众所周知，后两首歌出自周深的二专《反深代词》，都是类似于《流浪地球》的末日、前卫的主题，故第一首歌选择周深为《流浪地球 2》演唱的定义主题曲《人是_》，以保持主题的统一。

本次来不及做的内容：note 跟随背景音乐的节奏下落，即类似于音游中的谱。

6.1.5 UI 设计

《卡布不卡》中的 UI 如下：

- (1) 开始界面（登录界面）：玩家可通过滑动条调节音效、音乐的音量，有一个“开始游戏”的按钮。如图 6.1.2 所示。



图 6.1.2：开始界面

- (2) 关卡界面：背景是沧海校区的照片，中间为面板，左下方为摇杆，左侧为计分板。如图 6.1.3 所示。

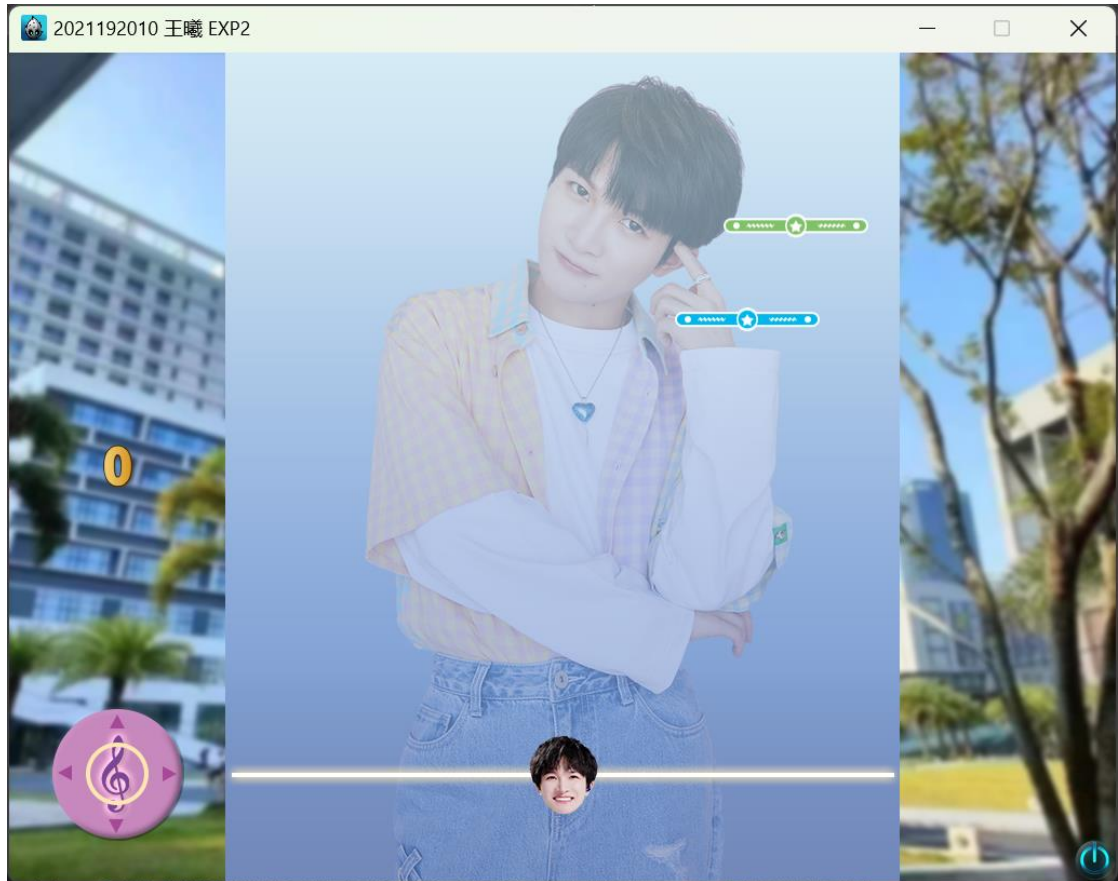


图 6.1.3: 关卡界面

(3) 关卡开始界面: 展示关卡条件、bgm、通关条件等信息。如图 6.1.4 所示。



图 6.1.4: 关卡开始界面

(4) 关卡结束界面: 展示准确率、Perfect 数、Good 数、Miss 数、Bad 数等信息, 右侧有重玩按钮和下一关按钮。如图 6.1.5 所示。



图 6.1.5: 关卡结束界面

6.1.6 特效设计

玩家接到 Note（即命中状态为 Perfect 或 Good）时会在接到的位置产生一个如图 6.1.6 所示的命中特效，该特效会一边旋转、一边放大、一边不断变透明直至消失。

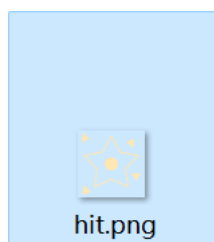


图 6.1.6: 命中特效

此外，还会在接到的位置产生一个如图 6.1.7 所示的命中状态的文字，该文字会一边上升、一边不断变透明直至消失。



图 6.1.7: 命中状态的文字

6.1.7 音效、音乐设计

《卡布不卡》的音效设计如下：

- (1) 开始界面中，玩家拖动滑动条调节音效、音乐音量时会播放周深的名场面“少管我”以指示音量大小。
- (2) 玩家接到 Note（即命中状态为 Perfect 或 Good）时会“噫”地一声。

《卡布不卡》的音乐设计如下：

- (1) 第一关：周深——《人是_》
- (2) 第二关：周深——《Intro》
- (3) 第三关：周深——《蜃楼》

6.2 设置 (Config.h, Config.cpp)

Config.h 中指定了面板大小、贴图路径、对象 tag、字体路径、UI 路径等信息。

此外，Config.cpp 实现了一个用于生成区间 $[l, r]$ 内的随机整数的函数 `int generateRandomInteger(int l, int r)`，其采用 `mt19937` 实现，以保证足够的随机性。

6.3 开始界面 (Login.h, Login.cpp)

6.3.1 UI 设计

在 Cocos Studio 的 UI 编辑器中设计开始界面，注意开启按钮的交互，并设置按钮的 tag。如图 6.3.1 所示。

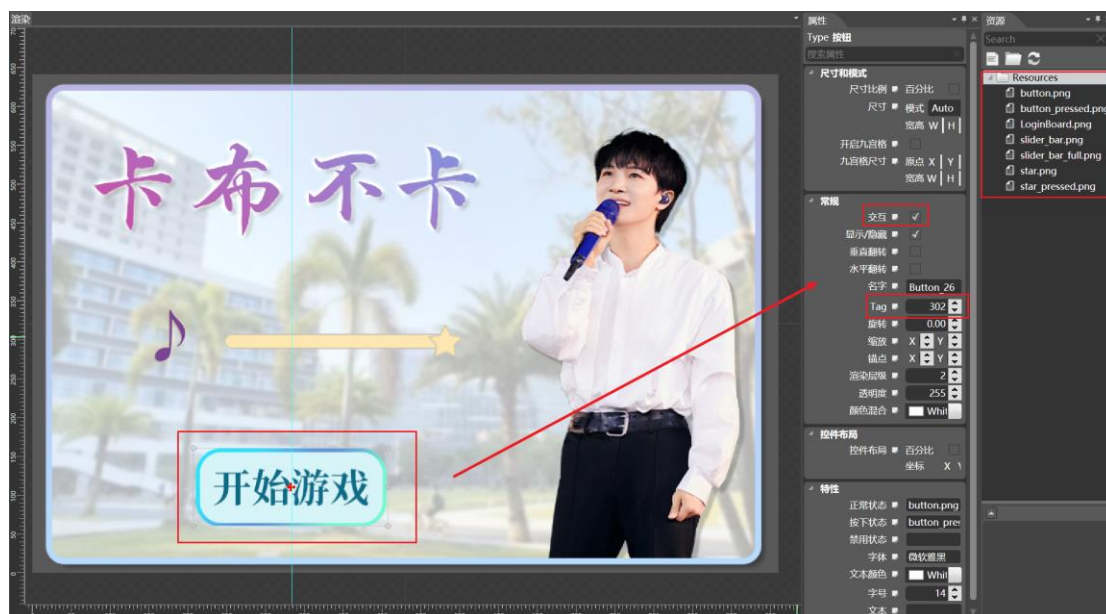


图 6.3.1：在 UI 编辑器中设计开始界面

设计完后 `Ctrl + E` 导出项目，指定导出路径后，选择“导出当前画布”、“导出使用大图”。如图 6.3.2 所示。

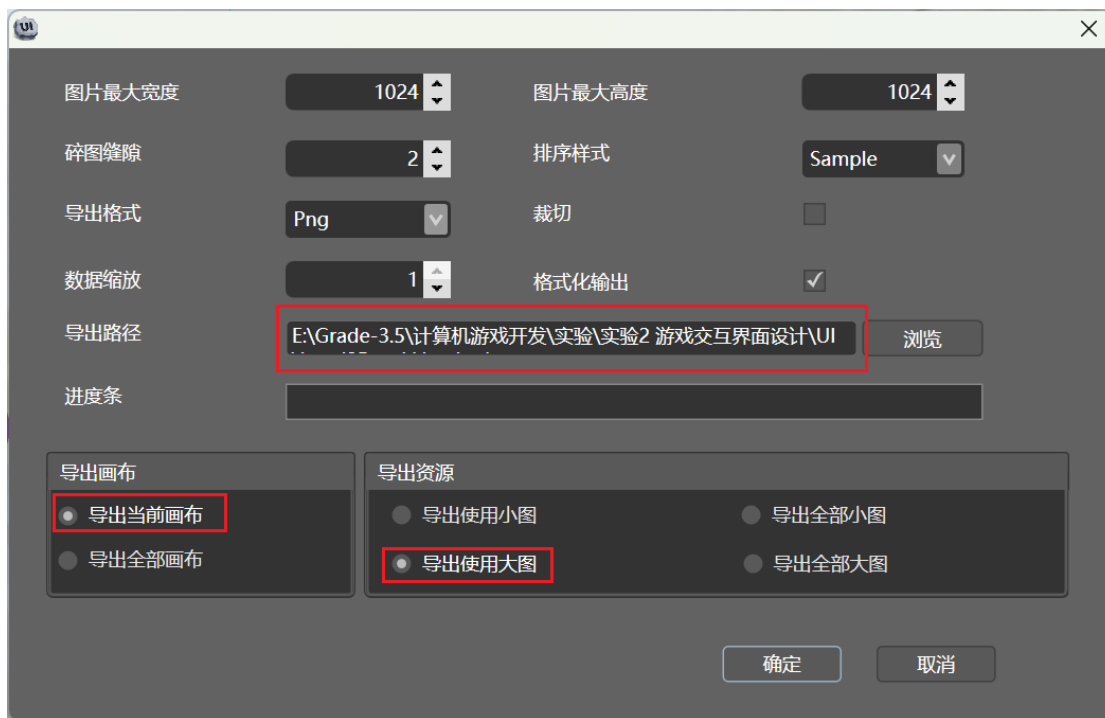


图 6.3.2: UI 编辑器导出设置

导出结果如图 6.3.3 所示。



图 6.3.3: Login 界面的导出结果

6.3.2 成员函数概览

Login.h 中定义了继承自 Layer 类的类 Login，其中除几个必须的函数外，还声明了加载背景函数、加载登录面板函数等函数。如图 6.3.4 所示。

```

34      // 加载背景函数
35      void loadBackground(std::string backgroundPath, int z);
36
37      // 加载登录面板
38      void loadLoginBoard(std::string loginBoardPath, int z);
39
40      // 滑动条事件回调函数
41      void sliderEvent(cocos2d::Ref* pSender, SliderEventType type);
42
43      // 预加载音效、音乐
44      void preloadSounds();
45
46      // 按钮事件回调函数
47      void buttonEvent(cocos2d::Ref* pSender);

```

图 6.3.4: Login.h 中声明的部分函数

下面将对部分函数的实现做详细说明。

6.3.3 加载登录面板

加载登录面板的函数 loadLoginBoard()从 UI 编辑器导出的 JSON 文件中读取 UI 并添加到场景中。注意 UI 的锚点未必位于中心，需根据实际情况调节摆放位置。为滑动条和按钮添加监听。如图 6.3.5 所示。

```
35 // 加载登录面板
36 void Login::loadLoginBoard(std::string loginBoardPath, int z) {
37     auto loginBoard = GUIReader::getInstance()->widgetFromFile(loginBoardPath.c_str());
38     auto position = loginBoard->getPosition();
39     loginBoard->setPosition(Vec2(position.x + 30, position.y + 80));
40     this->addChild(loginBoard);
41
42     // 滑动条监听
43     auto slider = (Slider*)loginBoard->getChildByTag(301);
44     slider->addEventListenerSlider(this, sliderpercentchangedselector(Login::sliderEvent));
45
46     // 按钮监听
47     auto button = (Button*)loginBoard->getChildByTag(302);
48     button->addTouchEventListeners(CC_CALLBACK_1(Login::buttonEvent, this));
49 }
```

图 6.3.5: loadLoginBoard()函数

其中滑动条事件的回调函数 sliderEvent()实现根据滑动条的值更改音效、音乐的音量，按钮事件的回调函数 buttonEvent()实现切换到场景 1，此处采用 pushScene()方法实现，无需处理边界问题。如图 6.3.6 所示。

```
97 void Login::sliderEvent(cocos2d::Ref* pSender, SliderEventType type) {
98     if (type == SLIDER_PERCENTCHANGED) {
99         Slider* slider = dynamic_cast<Slider*>(pSender);
100         VOLUME = slider->getPercent() / 100.0;
101         SimpleAudioEngine::getInstance()->setEffectsVolume(VOLUME);
102         SimpleAudioEngine::getInstance()->setBackgroundMusicVolume(VOLUME);
103         SimpleAudioEngine::getInstance()->playEffect(SHAOGUANWO_SOUND_PATH.c_str());
104     }
105 }
106
107 void Login::buttonEvent(cocos2d::Ref* pSender) {
108     level1Scene = Level1::createScene();
109     // Director::getInstance()->replaceScene(level1Scene);
110     Director::getInstance()->pushScene(level1Scene);
111 }
```

图 6.3.6: sliderEvent()函数、buttonEvent()函数

6.3.7 预加载音频

预加载音频函数 preloadSounds()加载了登录面板需用到的“少管我”音效。预加载音效有利于减少游戏播放音效时卡顿、掉帧的现象。如图 6.3.7 所示。

```
51 void Login::preloadSounds() {
52     // 预加载音效
53     SimpleAudioEngine::getInstance()->preloadEffect(SHAOGUANWO_SOUND_PATH.c_str());
54 }
```

图 6.3.7: preloadSounds()函数

6.4 时间、判定线、Note 类 (Note.h, Note.cpp)

6.4.1 时间类

时间类 Time 由分、秒、厘秒三个成员变量构成。如图 6.4.1 所示。

```

13  class Time {
14      private:
15          int min;
16          int s;
17          int cs; // 1 s = 100 cs

```

图 6.4.1: Time 类的成员变量

Time 类定义了时间换算、获取时间等函数，并重载了时间加减、比较等运算符。不再赘述。如图 6.4.2 所示。

```

24      // Time 换算成 cs
25      int getCs();
26
27      void setTime(int _min, int _s, int _cs);
28
29      // cs 换算成 Time
30      static Time getTime(int cs);
31
32      friend Time operator+(Time A, Time B);
33
34      friend Time operator-(Time A, Time B);
35
36      friend bool operator<(Time A, Time B);
37
38      friend bool operator>(Time A, Time B);
39
40      friend bool operator==(Time A, Time B);
41
42      Time operator++(int);

```

图 6.4.2: Time 类的成员函数

6.4.2 判定线类

判定线类 Line 包含判定线的精灵和窗口的大小，实现了设置判定线、获取判定线 y 坐标、更新判定线 y 坐标等函数。《卡布不卡》中判定线只能上下移动，故只涉及 y 坐标的变化。如图 6.4.3 所示。


```

45  class Line {
46      private:
47          Sprite* line;
48          Size visibleSize;
49
50      public:
51          Line();
52
53          ~Line();
54
55          void setLine(Sprite* _line);
56
57          Sprite* getLine();
58
59          float getY();
60
61          void updatePosition(float y);
62  };

```

图 6.4.3: Line 类

6.4.3 Note 类

Note 类包含精灵 `note`、到达判定线时间 `time`、对应的判定线 `line` 等成员变量。为每个 Note 指定对应的判定线有利于后期扩展，即场景中有多条不同方向的判定线，不同 `note` 朝不同的判定线下落，类似于 Phigros。如图 6.4.4 所示。

```

66  class Note {
67      private:
68          Sprite* note;
69          Time time; // 到达判定线的时间, 含预备时间
70          Line* line; // 对应的判定线
71          float width; // note 的宽度
72          float height; // note 的高度
73          float lastSpeed; // 上一次的速度
74          bool arrived; // 是否到达判定线
75          bool locked; // 是否被锁定
76          int timer; // 锁定计时器

```

图 6.4.4: Note 类的成员变量

Note 类除实现基本的设置、获取私有变量的函数外，还实现了检查命中状态的 `checkHit()` 函数、计算下落速度的 `calculateSpeed()` 函数、实现下落的 `fall()` 函数。如图 6.4.5 所示。

```

121 // 检查命中状态
122 int checkHit();
123
124 // 计算下落速度
125 float calculateSpeed(Time currentTime);
126
127 // 下落
128 void fall(float y);

```

图 6.4.5: Note 类的部分成员函数

checkHit()函数根据 6.1.3 评分机制中命中状态的判定返回 4 种命中状态之一。如图 6.4.6 所示。

```
181 // 0 为 Miss, 1 为 Perfect, 2 为 Good, 3 为 Bad
182 int Note::checkHit() {
183     // dis > 0 时 note 在判定线上方
184     float dis = getPosition().y - line->getY();
185
186     if (cmp(std::fabs(dis), height / 2) <= 0) { // Perfect
187         return 1;
188     }
189     else if (cmp(std::fabs(dis), height) <= 0) { // Good
190         return 2;
191     }
192     else if (dis > 0 && dis > height) { // Bad
193         return 3;
194     }
195     else { // Miss
196         return 0;
197     }
198     // assert(false);
199 }
```

图 6.4.6: checkHit()函数

calculateSpeed()函数和 fall()函数的实现简单，不再赘述。如图 6.4.7 所示。

```
201 float Note::calculateSpeed(Time currentTime) {
202     int timeDifference = (time - currentTime).getCs();
203     float yDifference = getPosition().y - line->getY();
204     return yDifference / timeDifference;
205 }
206
207 void Note::fall(float dy) {
208     note->setPosition(Point(getPosition().x, getPosition().y - dy));
209 }
```

图 6.4.7: calculateSpeed()函数和 fall()函数

6.5 关卡父类 (HelloWorldScene.h, HelloWorldScene.cpp)

6.5.1 抽象的必要性

不同关卡有很多共同点，如都需要加载资源的函数，都需要更新 note 状态的函数等。重复的实现不仅浪费代码，而且不易修改。为此，有必要抽象出一个关卡父类 HelloWorld，它实现所有关卡都需要的基本功能，并提供虚函数供继承 HelloWorld 的关卡类改写。

6.5.2 成员概览

HelloWorld 类包含下列成员变量。如图 6.5.1 所示。

```

72     protected:
73         Sprite* bean;
74         Size visibleSize;
75         Joystick* m_joystick;
76         Vector<Sprite*> ballVector;
77         bool pause;
78
79         int score;
80
81         int total_notes; // 总 note 数
82         int perfect_notes; // Perfect 的 note 数
83         int good_notes; // Good 的 note 数
84         int miss_notes; // Miss 的 note 数
85         int bad_notes; // Bad 的 note 数
86
87         std::vector<Label*> wordsTips; // 文本

```

图 6.5.1: HelloWorld 类的成员变量

HelloWorld 类除基本的函数外，还实现了下列的成员函数。如图 6.5.2 所示。

```

48     // 添加键盘监听
49     void addKeyboardListener();
50
51     // 键盘监听的回调函数
52     virtual void onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event);
53
54     // 加载提示
55     void loadHint(std::string hintPath, int tag, int z);
56
57     // 加载结果面板
58     void loadResultBoard(std::string resultBoardPath, int z);
59
60     // 重玩按钮回调函数
61     virtual void replayButtonEvent(cocos2d::Ref* pSender);
62
63     // 下一关按钮回调函数
64     virtual void nextButtonEvent(cocos2d::Ref* pSender);
65
66     // 展示分数
67     void displayScore();
68
69     // 展示文本
70     void displayWords(std::string words, int z);

```

图 6.5.2: HelloWorld 类的成员函数

6.5.3 加载结果面板函数

加载结果面板函数 loadResultBoard()类似于 Login 类中加载登录面板的函数，不再赘述。如图 6.5.3 所示。

```

263 void HelloWorld::loadResultBoard(std::string resultBoardPath, int z) {
264     auto resultBoard = GUIReader::getInstance()->widgetFromFile(resultBoardPath.c_str());
265     auto position = resultBoard->getPosition();
266     resultBoard->setPosition(Vec2(position.x + 30, position.y + 80));
267     this->addChild(resultBoard, z);
268
269     // 重玩按钮监听
270     auto replayButton = (Button*)resultBoard->getChildByName("Replay_Button");
271     replayButton->addTouchEventListener(CC_CALLBACK_1(HelloWorld::replayButtonEvent, this));
272
273     // 下一关按钮监听
274     auto nextButton = (Button*)resultBoard->getChildByName("Next_Button");
275     nextButton->addTouchEventListener(CC_CALLBACK_1(HelloWorld::nextButtonEvent, this));
276 }

```

图 6.5.3: loadResultBoard()函数

6.5.4 显示分数函数

显示分数函数 displayScore()展示玩家本关的准确率、Perfects 数等信息。注意准确率的分母不是总 note 数，否则会将游戏结束时仍在下落的 notes 也计入。如图 6.5.4 所示。

```

284 void HelloWorld::displayScore() {
285     std::string accuracy = std::to_string(perfect_notes + good_notes) + " / " + std::to_string(perfect_notes + good_notes + miss_notes);
286     auto accuracyTip = Label::createWithBMFont(TIPS_FNT_PATH, accuracy);
287     accuracyTip->setPosition(Point(visibleSize.width / 2 + 200, 475));
288     accuracyTip->setTag(ACCURACY_TAG);
289     this->addChild(accuracyTip, 11);
290
291     auto perfectTip = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(perfect_notes));
292     perfectTip->setPosition(Point(visibleSize.width / 2 + 200, 370));
293     perfectTip->setTag(PERFECT_TAG);
294     this->addChild(perfectTip, 11);
295
296     auto goodTip = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(good_notes));
297     goodTip->setPosition(Point(visibleSize.width / 2 + 200, 305));
298     goodTip->setTag(GOOD_TAG);
299     this->addChild(goodTip, 11);
300
301     auto missTip = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(miss_notes));
302     missTip->setPosition(Point(visibleSize.width / 2 + 200, 235));
303     missTip->setTag(MISS_TAG);
304     this->addChild(missTip, 11);
305
306     auto badTip = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(bad_notes));
307     badTip->setPosition(Point(visibleSize.width / 2 + 200, 170));
308     badTip->setTag(BAD_TAG);
309     this->addChild(badTip, 11);
310 }

```

图 6.5.4: displayScore()函数

6.5.5 修改计分板函数

修改计分板函数 modifyScore()先清除旧计分板（若有），更新当前得分，并将当前得分新建计分板。如图 6.5.5 所示。

```

20 void HelloWorld::modifyScore(int _score) {
21     // 清除旧计分板
22     auto trash = this->getChildByTag(SCOREBOARD_TAG);
23     if (trash) {
24         this->removeChildByTag(SCOREBOARD_TAG);
25     }
26
27     score = _score;
28     auto scoreboard = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(score));
29     scoreboard->setPosition(Vec2(100, visibleSize.height / 2));
30     scoreboard->setTag(SCOREBOARD_TAG);
31     this->addChild(scoreboard, 1);
32 }

```

图 6.5.5: modifyScore()函数

6.6 关卡二类（部分）(Level2.h, Level2.cpp)

此处忽略关卡一类的实现，因为关卡一只修改了关卡二中关于 Good 的判定。

6.6.1 成员概览

关卡二类 Level2 包含如图 6.6.1 所示的成员变量。

```
8      private:
9          Line hitLine; // 判定线
10         std::vector<Note> notes;
11         std::vector<Sprite*> hitEffects; // 命中特效
12         std::vector<Sprite*> hitTips; // 命中文字
13         Time currentTime;
14         std::vector<Note> trash; // 要删除的 notes
15         const int DELTA_TIMER; // 生成 note 的时间间隔
16         const Time END_TIME; // 结束时间
17         const int TARGET_SCORE; // 目标分
```

图 6.6.1: Level2 类的成员变量

Level2 类除实现基本的函数外，还实现了如图 6.6.2 和图 6.6.3 所示的成员函数。

```
35         // 加载判定线
36         void loadHitLine(std::string linePath, int y, int z);
37
38         // 加载 Note
39         void loadNote(int type, Time time, float x);
40
41         // 新建 Note
42         void createNote(float dt);
43
44         // 处理命中
45         void workHit(Note& note, int state);
46
47         // 更新 notes
48         void updateNotes(Time currentTime);
49
50         // 更新命中
51         void updateHits();
```

图 6.6.2: Level2 类的成员函数（1）

```
56         // 键盘监听的回调函数
57         void onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event);
58
59         // 显示结果
60         void displayResult();
61
62         // 重玩按钮回调函数
63         void replayButtonEvent(cocos2d::Ref* pSender);
64
65         // 下一关按钮回调函数
66         void nextButtonEvent(cocos2d::Ref* pSender);
67
68         // 更新文本
69         void updateWords();
```

图 6.6.3: Level2 类的成员函数 (2)

6.6.2 加载 note 函数

加载 note 函数 loadNote()根据 note 的种类、到达判定线的时间、初始 x 坐标加载 note。若 type = -1, 则为 3 种 note 种的随机一种。若 time = (0, 0, 0), 则为随机下落时间。若 x = -1, 则为随机初始 x 坐标。如图 6.6.4 所示。

```

34 // 加载 note : 种类、时间、初始 x 坐标
35 // type = -1 时为随机类型, time = (0, 0, 0) 时为随机时间, x = -1 时为随机初始 x 坐标
36 void Level2::loadNote(int type, Time time, float x) {
37     if (type == -1) {
38         type = generateRandomInteger(1, 3);
39     }
40
41     // 根据 note 的种类得到路径和 tag
42     std::string notePath;
43     int tag;
44     if (type == 1) {
45         notePath = NOTE1_PATH;
46         tag = NOTE1_TAG;
47     }
48     else if (type == 2) {
49         notePath = NOTE2_PATH;
50         tag = NOTE2_TAG;
51     }
52     else {
53         notePath = NOTE3_PATH;
54         tag = NOTE3_TAG;
55     }
56
57     // 新建 note
58     Note newNote;
59     auto note = Sprite::create(notePath);
60     newNote.setNote(note);
61
62     if (time == Time(0, 0, 0)) {
63         time.setTime(0, generateRandomInteger(1, 5), 0);
64     }
65
66     newNote.setTime(time);
67     newNote.setLine(&hitLine);
68     newNote.setWidth(note->getContentSize().width);
69     newNote.setHeight(note->getContentSize().height);
70     newNote.setLastSpeed(generateRandomInteger(3, 5));
71     newNote.setArrived(false);
72     newNote.setLocked(false);
73     newNote.setTimer(0);
74
75     if (!cmp(x, -1)) {
76         x = generateRandomInteger(
77             std::floor(BOARD_WIDTH_BIAS + newNote.getWidth() / 2),
78             std::ceil(BOARD_WIDTH_BIAS + BOARD_WIDTH - newNote.getWidth() / 2)
79         );
80     }
81
82     // 将新 note 加入场景
83     note->setPosition(Point(x, visibleSize.height));
84     this->addChild(note, 5);
85
86     // 将新 note 加入场景 notes 中
87     notes.push_back(newNote);
88 }

```

图 6.6.4: loadNote()函数

6.6.3 初始化函数

场景的初始化函数 `init()` 先加载背景、面板、判定线、关闭菜单、精灵、摇杆，并设置 `update()` 函数。如图 6.6.5 所示。

```
103 bool Level2::init() {
104     if (!Layer::init()) {
105         return false;
106     }
107
108     visibleSize = Director::getInstance()->getVisibleSize();
109     Vec2 origin = Director::getInstance()->getVisibleOrigin();
110
111     // 加载背景
112     loadBackground(BACKGROUND_PATH, 0);
113     // 加载面板
114     loadBoard(BOARD2_PATH, 1);
115     // 加载判定线
116     loadHitLine(LINE_PATH, 100, 2);
117
118     // add a "close" icon to exit the progress. it's an autorelease object
119     auto closeItem = MenuItemImage::create(
120         "CloseNormal.png",
121         "CloseSelected.png",
122         CC_CALLBACK_1(HelloWorld::menuCloseCallback, this));
123
124     closeItem->setPosition(Vec2(origin.x + visibleSize.width - closeItem->getContentSize().width / 2,
125         origin.y + closeItem->getContentSize().height / 2));
126
127     // create menu, it's an autorelease object
128     auto menu = Menu::create(closeItem, NULL);
129     menu->setPosition(Vec2::ZERO);
130     this->addChild(menu, 1);
131
132     // 移动精灵
133     bean = Sprite::create("Chapter10/bean.png");
134     bean->setPosition(Point(visibleSize.width / 2, 100));
135     this->addChild(bean, 6);
136
137     // 摇杆
138     m_joystick = Joystick::create(Vec2(100, 100), 63.0f, "Chapter10/j-btn.png", "Chapter10/j-bg.png");
139     this->addChild(m_joystick, 4);
140
141     this->scheduleUpdate();
142
143     // loadNote(-1, Time(0, 1, 0), -1);
144     // loadNote(-1, Time(0, 2, 0), -1);
145     // loadNote(-1, Time(0, 3, 0), -1);
146 }
```

图 6.6.5: `init()` 函数 (1)

`init()` 函数再初始化计分板的值、播放背景音乐、暂停游戏、暂停音乐、添加键盘监听、加载关卡的提示信息、重置计数器。如图 6.6.6 所示。


```

147 // 计分板
148 modifyScore(0);
149
150 // 背景音乐
151 SimpleAudioEngine::getInstance()->playBackgroundMusic(INTRO_PATH.c_str());
152
153 // 暂停游戏
154 pause = true;
155 SimpleAudioEngine::getInstance()->pauseBackgroundMusic();
156
157 // 添加键盘监听
158 addKeyboardListener();
159
160 // 加载提示信息
161 loadHint(LEVEL2_HINT_PATH, LEVEL2_HINT_TAG, 10);
162
163 // note 计数器
164 total_notes = 0;
165 perfect_notes = 0;
166 good_notes = 0;
167 miss_notes = 0;
168 bad_notes = 0;

```

图 6.6.6: init()函数 (2)

6.6.4 处理命中函数

处理命中函数 workHit()根据 note 的命中状态分别处理，处理方式如表 6.6.1 所示。

表 6.6.1: note 的命中状态对应的效果

命中状态 \ 效果	文字信息	特效	音效	得分	计数	删除 note	上锁
Miss	√				√	√	
Perfect	√	√	√	+2	√	√	
Good	√	√	√	+1	√	√	
Bad	√			-1	√		√

以 Perfect 的处理为例，如图 6.5.7 所示。

```

193 case 1: { // Perfect
194     // 文字
195     auto perfect = Sprite::create(PERFECT_PATH);
196     perfect->setPosition(position);
197     this->addChild(perfect, 8);
198     hitTips.push_back(perfect);
199
200     // 效果
201     auto hit = Sprite::create(HIT_PATH);
202     hit->setPosition(position);
203     this->addChild(hit, 7);
204     hitEffects.push_back(hit);
205
206     // 音效
207     SimpleAudioEngine::getInstance()->playEffect(HIT_SOUND_PATH.c_str());
208
209     // 得分
210     modifyScore(score + 2);
211
212     // 计数
213     perfect_notes++;
214
215     trash.push_back(note);
216
217     break;
218 }

```

图 6.6.7: workHit()函数中 Perfect 的处理

进入 workHit()函数时, 先对 note 上锁, 如图 6.6.8 所示。对 Miss、Perfect、Good 的情况, 是否上锁不影响结果, 因为出现这些状态都会将 note 删除。但 Bad 的情况不会删除 note, 此时玩家与 note 碰撞产生 Bad 时可能会在短时间内产生多次 Bad。为此, 先将判 Bad 的 note 上锁, 并用一个计时器延迟一定时间后再解锁。后续的 update()函数中只对未上锁的 note 做碰撞检测。

```

173 // 0 为 Miss, 1 为 Perfect, 2 为 Good, 3 为 Bad
174 void Level2::workHit(Note& note, int state) {
175     Point position = note.getPosition();
176     note.setLocked(true);

```

图 6.6.8: 进入 workHit()函数时, 先对 note 上锁

6.6.5 显示结果函数

显示结果函数 displayResult()先加载关卡结果的 UI, 并调用父类 HelloWorld 中的 displayScore()函数展示玩家的数据。如图 6.6.9 所示。

```

352 void Level2::displayResult() {
353     loadResultBoard(LEVEL2_RESULT_PATH, 10);
354     displayScore();
355 }

```

图 6.6.9: displayResult()函数

其中结果界面的 UI 用 Cocos Studio 的 UI 编辑器绘制, 注意开启按钮的交互。此处将重玩按钮、下一关按钮的名称分别设置为 “Replay_Button” 和 “Next_Button” 以供调用。如图 6.6.10 所示。实践表明: 该方法比指定 tag 更方便。



图 6.6.10: 用 UI 编辑器绘制结果界面的 UI

6.6.6 键盘交互回调函数

键盘交互回调函数 `onKeyPressed()` 处理键盘事件。如图 6.6.11 所示。

《卡布不卡》中的键盘交互如下：

- (1) ESC：关闭游戏。
- (2) 空格：关闭关卡前提示信息，开始游戏。
- (3) P：暂停游戏和音乐。
- (4) K（调试用）：跳关。

```

416 void Level2::onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event) {
417     switch (keyCode) {
418         case EventKeyboard::KeyCode::KEY_ESCAPE: { // ESC: 关闭
419             Director::getInstance()->end();
420             break;
421         }
422         case EventKeyboard::KeyCode::KEY_SPACE: { // 空格: 开始游戏
423             this->removeChildByTag(LEVEL2_HINT_TAG);
424             pause = false;
425             SimpleAudioEngine::getInstance()->resumeBackgroundMusic();
426             break;
427         }
428         case EventKeyboard::KeyCode::KEY_P: { // P: 暂停
429             pause ^= 1;
430             if (pause) {
431                 // 背景音乐
432                 SimpleAudioEngine::getInstance()->pauseBackgroundMusic();
433             }
434             else {
435                 // 背景音乐
436                 SimpleAudioEngine::getInstance()->resumeBackgroundMusic();
437             }
438             break;
439         }
440         case EventKeyboard::KeyCode::KEY_K: { // K: 下一关
441             Director::getInstance()->popScene();
442             level3Scene = Level3::createScene();
443             Director::getInstance()->pushScene(level3Scene);
444             break;
445         }
446     }
447 }

```

图 6.6.11: onKeyPressed()函数

6.6.7 按钮事件回调函数

结果界面中有两个按钮事件回调函数。如图 6.6.12 所示。

(1) 重玩按钮事件回调函数 replayButtonEvent()弹出当前场景栈的栈顶场景，新建当前关卡的场景的对象并重新入栈，以达到重玩当前关卡的效果。

(2) 下一关按钮事件回调函数 nextButtonEvent()弹出当前场景栈的栈顶场景，新建下一关卡的场景的对象并入栈，以达到下一关的效果。

```

449 void Level2::replayButtonEvent(cocos2d::Ref* pSender) {
450     Director::getInstance()->popScene();
451     level2Scene = Level2::createScene();
452     Director::getInstance()->pushScene(level2Scene);
453 }
454
455 void Level2::nextButtonEvent(cocos2d::Ref* pSender) {
456     Director::getInstance()->popScene();
457     level3Scene = Level3::createScene();
458     Director::getInstance()->pushScene(level3Scene);
459 }

```

图 6.6.12: 两个按钮事件回调函数

6.7 update()函数 (Level2.h, Level2.cpp)

因 update()函数较复杂，故单独用一节说明。

6.7.1 更新文本

关卡结束后，结果页面上方会有淡出的胜负判定的文本。如图 6.7.1 和图 6.7.2 所示。



弹出结果页面时场景处于暂停状态，但不影响文本的淡出。故 `update()` 函数先调用更新文本函数 `updateWords()` 以更新文本，再判定场景是否暂停。如图 6.7.3 所示。

```
357 void Level2::update(float dt) {  
358     updateWords();  
359  
360     if (pause) {  
361         return;  
362     }
```

图 6.7.3: `update()` 函数先调用 `updateWords()` 函数再判断暂停

6.7.2 碰撞检测

`update()` 函数对未上锁的 `note` 做碰撞检测。若碰撞，则获取碰撞状态并调用 `workHit()` 函数处理。如图 6.7.4 所示。

```
364 // 碰撞检测  
365 for (auto& note : notes) {  
366     if (note.getLocked()) {  
367         continue;  
368     }  
369  
370     if (note.getNote()->getBoundingBox().intersectsRect(bean->getBoundingBox())) {  
371         int state = note.checkHit();  
372         workHit(note, state);  
373     }  
374 }
```

图 6.7.4: 碰撞检测

6.7.3 更新 notes

`update()` 函数调用 `updateNotes()` 函数更新 `notes`。如图 6.7.5 所示。

```
376 // 更新 notes  
377 updateNotes(currentTime);
```

图 6.7.5: `update()` 函数调用 `updateNotes()` 函数

其中 `updateNotes()` 函数先比较当前时间与 `note` 的到达时间以判定 `note` 是否到达判定线，若 `note` 已到达判定线且未被 6.7.2 碰撞检测处理，则视 `note` 与判定线间的距离判 Miss。到达判定线后的 `note` 以恒定速度下落，直至判 Miss 后消失，这采用 `note` 的 `lastSpeed` 变量实现。对未到达判定线的 `note`，调用 `calculateSpeed()` 函数根据它到判定线的距离和剩余的时间

更新速度。最后删除 notes。如图 6.7.6 所示。

```
283 void Level2::updateNotes(Time currentTime) {
284     for (auto& note : notes) {
285         // note 已到判定线
286         if (currentTime > note.getTime() || currentTime == note.getTime()) {
287             note.setArrived(true);
288             float dis = note.getLine()->getY() - note.getPosition().y;
289             if (cmp(dis, note.getHeight()) > 0) { // Miss
290                 workHit(note, 0);
291                 continue;
292             }
293         }
294
295         // note 到判定线后以恒定速度下落
296         float speed = note.getLastSpeed();
297         if (!note.getArrived()) {
298             speed = note.calculateSpeed(currentTime);
299         }
300         note.fall(speed);
301         note.setLastSpeed(speed);
302
303         note.updateTimer();
304     }
305
306     // 删除 notes
307     for (auto& note : trash) {
308         this->removeChild(note.getNote());
309         notes.erase(find(notes.begin(), notes.end(), note));
310     }
311     trash.clear();
312 }
```

图 6.7.6: updateNotes()函数

其中 note.updateTimer()函数更新上锁的 note 的计时器，达到一定时间解锁。如图 6.7.7 所示。

```
166 void Note::updateTimer() {
167     if (!locked) {
168         return;
169     }
170
171     if (++timer >= MAX_TIMER) {
172         timer = 0;
173         locked = false;
174     }
175 }
```

图 6.7.7: note.updateTimer()函数

6.7.4 更新命中效果

update()函数调用 updateHits()函数更新命中效果。如图 6.7.8 所示。

```
379 // 更新命中效果
380 updateHits();
```

图 6.7.8: update()函数调用 updateHits()函数

其中 updateHits()函数更新命中特效和命中文字，如图 6.7.9 和图 6.7.10 所示，其中的参数在 Config.h 中指定，如图 6.7.11 所示。

- (1) 命中特效：一边放大、一边旋转、一边淡出。
- (2) 命中文字：一边上升、一边淡出。

```

314 void Level2::updateHits() {
315     // 命中特效
316     std::vector<Sprite*> trashEffects;
317     for (auto& effect : hitEffects) {
318         float scale = effect->getScale();
319         effect->setScale(scale + DELTA_SCALE);
320         float angle = effect->getRotation();
321         effect->setRotation(angle + DELTA_ANGLE);
322         float opacity = effect->getOpacity();
323         effect->setOpacity(opacity - DELTA_OPACITY);
324
325         if (effect->getOpacity() < 75) {
326             trashEffects.push_back(effect);
327         }
328     }
329     for (auto& effect : trashEffects) {
330         this->removeChild(effect);
331         hitEffects.erase(find(hitEffects.begin(), hitEffects.end(), effect));
332     }
}

```

图 6.7.9: updateHits()函数更新命中特效

```

334 // 命中文字
335 std::vector<Sprite*> trashTips;
336 for (auto& tip : hitTips) {
337     Point position = tip->getPosition();
338     tip->setPosition(Point(position.x, position.y + DELTA_POSITION_Y));
339     float opacity = tip->getOpacity();
340     tip->setOpacity(opacity - DELTA_OPACITY);
341
342     if (tip->getOpacity() < 75) {
343         trashTips.push_back(tip);
344     }
345 }
346 for (auto& tip : trashTips) {
347     this->removeChild(tip);
348     hitTips.erase(find(hitTips.begin(), hitTips.end(), tip));
349 }
350 }

```

图 6.7.10: updateHits()函数更新命中文字

```

53 // note 命中效果
54 static std::string HIT_PATH = "hit.png";
55 static std::string PERFECT_PATH = "perfect.png";
56 static std::string GOOD_PATH = "good.png";
57 static std::string BAD_PATH = "bad.png";
58 static std::string MISS_PATH = "miss.png";
59 const float DELTA_SCALE = 0.025; // 每次 scale += 0.025
60 const float DELTA_ANGLE = 1; // 每次转 1°
61 const float DELTA_OPACITY = 2; // 每次 -2% 不透明度
62 const float DELTA_POSITION_Y = 1; // 每次 y += 1

```

图 6.7.11: Config.h 中指定特效参数

变化效果如图 6.7.12 所示。



图 6.7.12: 命中特效和文字

6.7.5 控制周深移动

与《贪食豆》完全相同，不再赘述。如图 6.7.13 所示。

```

382 // 控制周深移动
383 auto direction = m_joystick->getDirection(); // 摇杆移动方向
384 auto destination = bean->getPosition() + direction * m_joystick->getVelocity() * 0.15; // 预期目标点
385 auto beanSize = bean->getContentSize(); // 贪食豆大小
386 destination.x = std::max(destination.x, BOARD_WIDTH_BIAS + beanSize.width / 2);
387 destination.x = std::min(destination.x, BOARD_WIDTH_BIAS + BOARD_WIDTH - beanSize.width / 2);
388 destination.y = std::max(destination.y, 0 + beanSize.height / 2);
389 destination.y = std::min(destination.y, BOARD_HEIGHT - beanSize.height / 2);
390 bean->setPosition(destination);

```

图 6.7.13: 控制周深移动

6.7.6 生成 note

update()函数中先更新当前时间 currentTime，并每隔一定时间调用 createNote()函数随机生成 note。如图 6.7.14 所示。时间间隔 DELTA_TIMER 在 Config.h 中指定。

```

392 currentTime++;
393
394 // 生成 note
395 if (currentTime.getCs() % DELTA_TIMER == 0) {
396     createNote(0.1f);
397 }

```

图 6.7.14: update()更新 currentTime 并调用 createNote()函数

6.7.7 胜负判定

update()函数中根据当前分数是否达到关卡的目标分数，或当前时间是否超过关卡限制时间进行胜负判定，注意判定 All Perfect 时的 note 总数不是 total_notes。判定胜负后，调用 displayResult()函数显示结果，并调用 displayWords()函数显示胜负状态。如图 6.7.15 所示。

```

399 // 胜负判定
400 if (score >= TARGET_SCORE || currentTime > END_TIME) {
401     pause = true;
402     SimpleAudioEngine::getInstance()->pauseBackgroundMusic();
403     displayResult();
404
405     if (score >= TARGET_SCORE) {
406         if (perfect_notes == perfect_notes + good_notes + miss_notes) {
407             displayWords("ALL PERFECT!", 20);
408         }
409         else {
410             displayWords("SUCCESS!", 20);
411         }
412     }
413     else {
414         displayWords("GAME OVER!", 20);
415     }
416 }
417 }

```

图 6.7.15: update()函数的胜负判定

6.8 关卡三类（部分）(Level3.h, Level3.cpp)

关卡三类 Level3 的实现绝大部分与关卡二类 Level2 一致，仅有少量部分需要修改。

6.8.1 判定线移动

Level3 类新增三个成员变量以控制判定线移动，分别为判定线的最小 y 值、最大 y 值和移动时间间隔。如图 6.8.1 所示。

```

7 class Level3 : public HelloWorld {
8     private:
9         Line hitLine; // 判定线
10        std::vector<Note> notes;
11        std::vector<Sprite*> hitEffects; // 命中特效
12        std::vector<Sprite*> hitTips; // 命中文字
13        Time currentTime;
14        std::vector<Note> trash; // 要删除的 notes
15        const int DELTA_TIMER; // 生成 note 的时间间隔
16        const Time END_TIME; // 结束时间
17        const int TARGET_SCORE; // 目标分
18
19        const int MIN_LINE_Y;
20        const int MAX_LINE_Y;
21        const int MOVE_TIMER;

```

图 6.8.1: Level3 类新增的成员变量

update()函数中每隔一段时间先随机上下移动判定线，再更新 notes 和命中。如图 6.8.2 所示。

```

381 // 随机移动判定线 (先)
382 if (currentTime.getCs() % DELTA_TIMER == 0) {
383     int direction = generateRandomInteger(0, 1);
384     int distance = generateRandomInteger(5, 20);
385     float targetY = hitLine.getY() + (direction ? distance : -distance);
386     targetY = std::max(targetY, (float)MIN_LINE_Y);
387     targetY = std::min(targetY, (float)MAX_LINE_Y);
388     hitLine.updatePosition(targetY);
389 }
390
391 // 更新 notes (后)
392 updateNotes(currentTime);
393 updateHits();

```

图 6.8.2: update()函数中随机上下移动判定线

6.8.2 下一关

因关卡三是最后一关，故在其结果页面中点击下一关可回到开始界面。修改下一关按钮事件的回调函数 nextButtonEvent(), 如图 6.8.3 所示。

```

471 void Level3::nextButtonEvent(cocos2d::Ref* pSender) {
472     Director::getInstance()->popScene();
473     loginScene = Login::createScene();
474     Director::getInstance()->pushScene(loginScene);
475 }

```

图 6.8.3: 下一关按钮事件的回调函数

6.9 运行效果

运行效果、详细说明见【附件】卡布不卡.mp4。

四、实验结论或心得体会

7. 实验结论

《贪食豆》的运行效果见【附件】贪食豆.mp4。

《卡布不卡》的运行效果、详细说明见【附件】卡布不卡.mp4。

7.1 编译游戏

编译并运行项目，成功运行游戏。如图 7.1.1 所示。

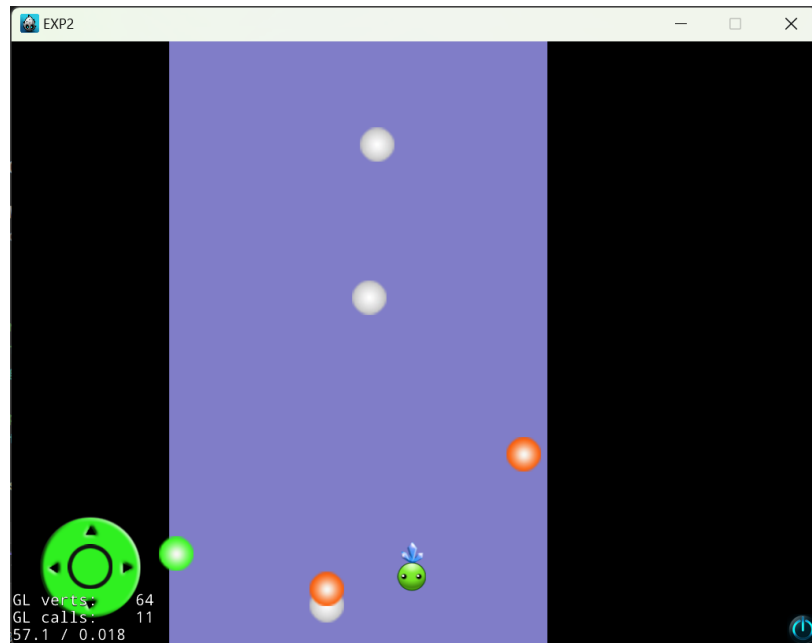


图 7.1.1：成功运行游戏

7.2 修改标题

成功修改标题为“2021192010 王曦 EXP2”。如图 7.2.1 所示。

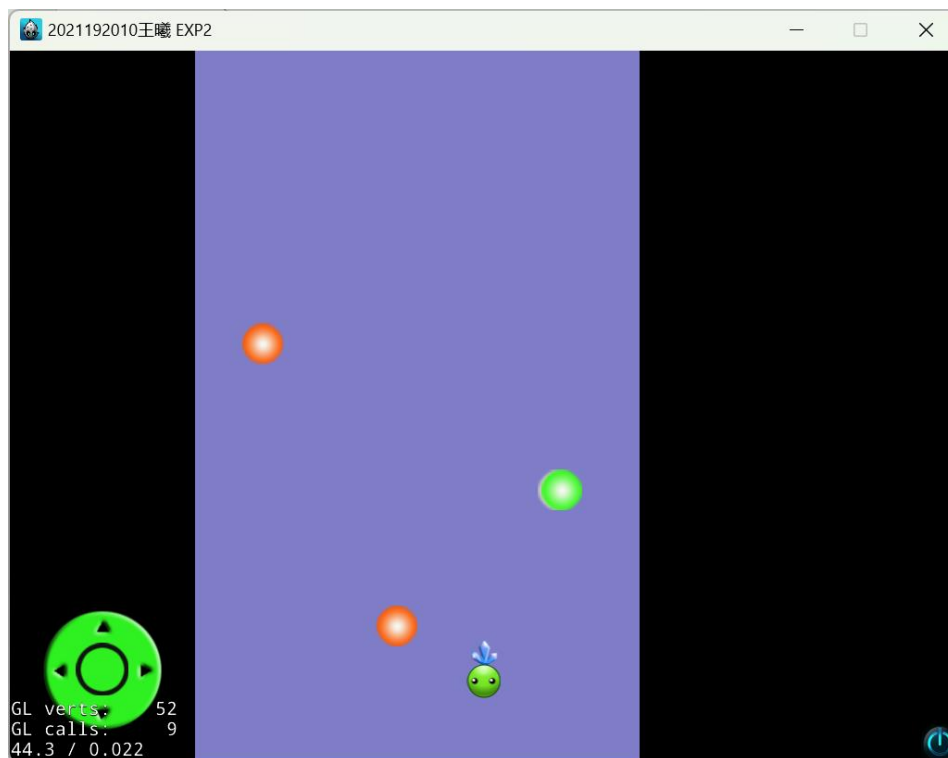


图 7.2.1: 修改标题的结果

7.3 扩展摇杆功能

将摇杆的功能修改为根据拉动方向 360° 地移动贪食豆。

运行效果见【附件】贪食豆.mp4。

7.4 计分板

在面板左侧创建计分板，并根据接到的球的种类赋不同的得分。如图 7.4.1 和图 7.4.2 所示。



图 7.4.1: 增加计分板

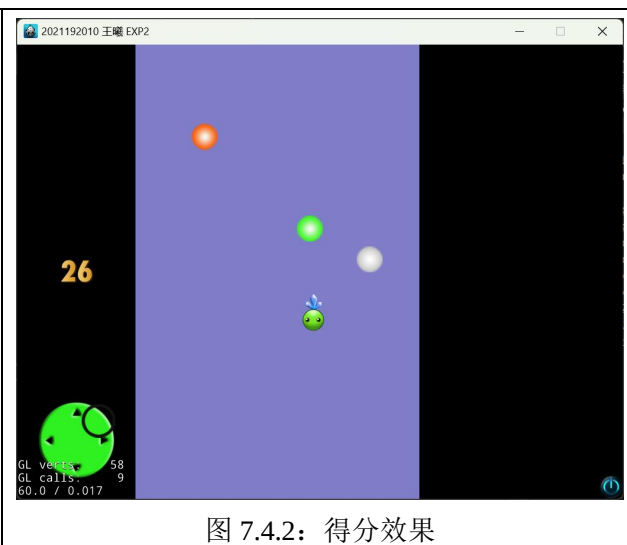


图 7.4.2: 得分效果

运行效果见【附件】贪食豆.mp4。

7.5 开始界面

用 Cocos Studio 的 UI 编辑器绘制了开始界面，包含调节音效、音乐音量的滑动条和开始游戏按钮。如图 7.5.1 所示。



图 7.5.1：开始界面

运行效果见【附件】卡布不卡.mp4。

7.6 重玩功能

每个关卡的结果界面有重玩按钮，点击可重玩该关卡。如图 7.6.1 所示。



图 7.6.1: 重玩按钮

运行效果见【附件】卡布不卡.mp4。

7.7 增加关卡

《卡布不卡》设三个关卡，难度递增。详情见 6.1.4 关卡设置。

运行效果见【附件】卡布不卡.mp4。

7.8 优化

7.8.1 《贪食豆》Bug 修复

修复了《贪食豆》中球越界、球一闪而过、摇杆判定范围的 Bug。

运行效果见【附件】贪食豆.mp4。

7.8.2 《卡布不卡》

将《贪食豆》中的贪食豆、豆子分别替换为周深、note，加入了接 note 准确度的判定，设置 3 个关卡，加入了特效、音效等，完成了周深接音符小游戏《卡布不卡》。

运行效果见【附件】卡布不卡.mp4。

8. 实验心得

8.1 核心心得

- (1) 做游戏最花时间的部分不是写代码，而是策划、制作音乐和美术资源、测试。
- (2) cocos-2dx 不好用。
- (3) 好像敲代码敲出肌腱炎了。

8.2 其它心得

本次实验中，我通过学习 Cocos2d-x 游戏引擎的使用，深入了解了交互界面设计的原理和方法，并掌握了 Cocos2d-x 中的用户交互、触摸事件、碰撞检测等机制。

通过本次实验，我不仅学会了交互界面设计的基本原理和方法，还掌握了使用 Cocos2d-x 进行游戏开发的基本技能。我相信这些知识和技能对我今后的游戏开发之路会有很大的帮助，我会继续努力学习，不断提升自己的技术水平，成为一名优秀的游戏开发工程师。

指导教师批阅意见：

成绩评定：

评语：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。